Projekt zaliczeniowy

Zaawansowane programowanie w języku Python
Rok akademicki 2025/2026

Autorzy:
Wiktoria Bąk
Kinga Maciasz

Projekt obiektowy w aplikacji Streamlit

Analiza The Movies Dataset – dokumentacja

Opis projektu

Projekt jest interaktywną aplikacją webową stworzoną w Pythonie przy użyciu biblioteki Streamlit. Aplikacja służy do eksploracji i analizy zbioru danych *The Movies Dataset* (Kaggle[1]), zawierającego informacje o filmach.

Celem projektu jest:

- podstawowa eksploracja danych filmowych,
- wizualizacja trendów (liczba filmów, średnie oceny),
- analiza popularności gatunków,
- klasteryzacja filmów na podstawie budżetu i przychodu oraz popularności i oceny,
- wybór najlepiej ocenianych filmów według roku i gatunków.

Aplikacja działa lokalnie w przeglądarce jako strona Streamlit.
W aktualnej wersji projekt został przebudowany z funkcyjnego na obiektowy.

[1] https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset

Struktura projektu

Projekt został zaprojektowany zgodnie z zasadami programowania obiektowego. Logika aplikacji została podzielona na wyspecjalizowane klasy, z których każda odpowiada za jeden obszar odpowiedzialności. Poszczególne pliki Pythona odpowiadają pokrótce za:

- *main.py* – punkt wejścia aplikacji, uruchamia obiekt aplikacji
- *movie_app.py* – główny plik aplikacji Streamlit (interfejs użytkownika)
- *loader.py* – pobieranie i wczytywanie danych z Kaggle lub lokalnego pliku CSV
- *processors.py* – przetwarzanie i czyszczenie danych
- *analiza.py* – analizy statystyczne i agregacje danych
- *cluster.py* – klasteryzacja danych filmowych
- *decorators.py* – dekoratory pomocnicze

Wymagania początkowe

Wymagania

- Python 3.9+ (zalecane)
- Konto na Kaggle i plik *kaggle.json* (opcjonalnie, jeśli dane mają być pobierane automatycznie)

Kaggle API (opcjonalne)

Aby umożliwić automatyczne pobieranie danych:

1. Pobierz plik *kaggle.json*
2. Umieść go w katalogu:
    - Windows: *C:\Users\\<użytkownik>\\.kaggle*
    - Linux/Mac: *~/.kaggle/*

Jeśli Kaggle API nie będzie dostępne, aplikacja spróbuje użyć lokalnego pliku *data/movies_metadata.csv.* Logika ta jest zaimplementowana w klasie *KaggleMoviesLoader.*

Uruchamianie aplikacji

Plik *main.py* pełni rolę punktu startowego – pozwala na oddzielenie logiki uruchamiana od logiki aplikacyjnej. Jego jedynym zadaniem jest utworzenie instancji klasy *MovieApp* oraz uruchomienie aplikacji:

```
from src.movie_app import MovieApp


def main():
    app = MovieApp()
    app.run()


if __name__ == "__main__":
    main()
```

W celu otwarcia strony w terminalu projektu należy uruchomić polecenie:

```
streamlit run main.py
```

Po chwili aplikacja otworzy się w przeglądarce (domyślnie pod adresem http://localhost:8501).

Klasa *MovieApp* – główny kontroler aplikacji

Plik: *movie_app.py*

Klasa *MovieApp* zarządza:

- Konfiguracją Streamlit
- Wczytywaniem danych
    - Dane są przechowywane w *st.session_state*, aby nie były wczytywane ponownie przy każdej interakcji użytkownika

```
def load_data(self):
    if "df" not in st.session_state:
        loader = KaggleMoviesLoader()
        st.session_state.df = loader.load()
```

- Przygotowaniem danych

```
def clean_data(self):
    self.df_clean = (
        MoviePreprocessor(self.df)
        .extract_year()
        .filter_recent()
        .get_df()
    )
```

- Zakładkami interfejsu użytkownika
- Przekazywaniem danych do modułów analitycznych.

Zakładki aplikacji

Zakładka „Podstawowa eksploracja"

Służy do zapoznania się ze strukturą danych oraz podstawowych rozkładów zmiennych numerycznych i kategorycznych.

Funkcjonalności:

- Wyświetlanie listy kolumn występujących w zbiorze danych

```
for col_name in df.columns:
    st.write(col_name)
```

- Opis kolumn na podstawie słownika *COLUMN_DESCRIPTIONS*

```
desc = COLUMN_DESCRIPTIONS.get(col, "Brak opisu")
```

- Podgląd pierwszych wierszy danych

```
st.dataframe(df.head(5)
```

- Analiza kolumn numerycznych:
  - wybór kolumny numerycznej

```
num col = st.selectbox("Wybierz kolumnę numeryczną", numeric cols)
```

  - konwersja danych i usunięcie braków

```
col data = pd.to numeric(df[num col], errors='coerce').dropna()
```

  - statystyki opisowe

```
col_data.describe()[["mean", "std", "min", "max"]]
```

  - histogram wartości z użyciem Plotly
- Analiza kolumn kategorycznych:
  - Spłaszczenie danych JSON-owych

```
cat_data = preprocessor.extract_categorical_values(cat_col)
```

  - zliczanie wystąpień (*Counter(cat_data)*)
  - interaktywny wykres słupkowy stworzony z użyciem Plotly

Zakładka „Filtry i wykresy"

Umożliwia dynamiczne filtrowanie danych oraz obserwację trendów w czasie.

Funkcjonalności:

- Ustawienie minimalnego roku wydania filmu

```
df_filtered = df[df["year"] >= year_slider]
```

- Ustawienie minimalnej oceny filmu

```
df filtered = df filtered[df filtered["vote average"] >= rating slider]
```

- Filtrowanie tytułów po nazwie

```
df_filtered = df_filtered[df_filtered["title"] \
    .str.contains(title_filter, case=False, na=False)]
```

- Wybór liczby najpopularniejszych gatunków do wyświetlenia

```
analyzer.top_genres(top_genre_count))
```

Wyświetlane wykresy:

- Liczba filmów w poszczególnych latach (*analyzer.movies_per_year()*),
- Średnia ocena filmów na rok (*analyzer.average_rating_per_year()*),
- Najpopularniejsze gatunki filmowe (*analyzer.top_genres()*).

Zakładka „Klasteryzacja"

Zakładka pozwala grupować filmy na podstawie wybranych cech.

Implementacja:

```
clusterer = BudgetRevenueCluster(df)\
    .preprocess(remove_zero)\
    .run_kmeans(n_clusters)
```

lub

```
clusterer = PopularityRatingCluster(df)\
    .preprocess(remove_zero)\
    .run_kmeans(n_clusters)
```

Funkcjonalności:

- Wybór typu klasteryzacji
    - budżet vs przychód
    - popularność vs ocena
- Wybór liczby klastrów,
- Możliwość usunięcia filmów z zerowymi wartościami zmiennych.

Wynik:

- Interaktywny wykres punktowy (Plotly) z podziałem na klastry.

Zakładka „Najlepiej oceniane filmy"

Pozwala na wybór najlepszych filmów według roku i gatunku (lub kilku gatunków).

Funkcjonalności:

- Wybór konkretnego roku (*st.selectbox("Wybierz rok", years)*)
- Wybór jednego lub wielu gatunków, (*st.multiselect("Wybierz gatunek(i)", flat_genres)*)
- Filtrowanie po liczbie głosów (*df_filtered[df_filtered["vote_count"] >= 100]*)
- Sortowanie i wybór top 10 filmów

```
df_filtered.sort_values(
    ["vote_average", "vote_count"],
    ascending=[False, False]
).head(10)
```

Moduły pomocnicze projektu

analiza.py

Zawiera klasę odpowiedzialną za analizę statystyczną danych filmowych oraz agregację informacji wykorzystywanych w wizualizacjach.

Klasa MovieAnalyzer

Odpowiedzialności:

- obliczenia agregujące dane według roku
- analiza ocen filmów
- analiza popularności gatunków filmowych

Najważniejsze metody:

- *movies_per_year()* – zwraca liczbę filmów wydanych w poszczególnych latach
- *average_rating_per_year()* – oblicza średnią ocenę filmów dla każdego roku
- *top_genres(top_n)* – zwraca słownik najczęściej występujących gatunków filmowych

processors.py

Zawiera odpowiada za czyszczenie, transformację i przygotowanie danych przed analizą.

Klasa MoviePreprocessor

Odpowiedzialności:

- przygotowanie danych czasowych
- filtrowanie rekordów
- przetwarzanie danych kategorycznych zapisanych w formacie JSON

Najważniejsze metody:

- *extract_year()* – konwertuje kolumnę *release_date* na typ daty i tworzy nową kolumnę *year*
- *filter_recent(year_threshold)* – filtruje dane według minimalnego roku wydania
- *extract_categorical_values(column)* – spłaszcza dane kategoryczne zapisane jako listy słowników
- *get_dt()* – zwraca przetworzoną ramkę danych

loader.py

Odpowiada za ładowanie danych do aplikacji oraz obsługę ich różnych źródeł.

Klasa KaggleMoviesLoader

Odpowiedzialności:

- pobieranie danych z serwisu Kaggle przy użyciu Kaggle API
- obsługa błędów uwierzytelnienia
- wczytywanie lokalnego pliku CSV jako rozwiązania awaryjnego

Najważniejsza metoda:

- *load()* – odpowiada za wszystko, co opisano powyżej, w dodatku wykorzystując dekoratory do ładowania danych

decorators.py

Zawiera dekoratory wykorzystywane w aplikacji do poprawy czytelności interfejsu oraz monitorowania wydajności.

Dostępne dekoratory:

- *measure_time* – mierzy czas wykonania – informacja wypisywana jest w konsoli
- *with_spinner(text)* – informuje użytkownika o procesie załadowywania danych za pomocą animacji kręcącego się kółka

cluster.py

Realizuje klasteryzację danych przy użyciu algorytmu KMeans. Zastosowano dziedziczenie, co umożliwia łatwe rozszerzanie o nowe typy klasteryzacji.

Klasa bazowa BaseCluster

Odpowiedzialności:

- standaryzacja danych
- uruchomienie algorytmu KMeans
- przypisanie etykiet klastrów do danych

Najważniejsze metody:

- *fit_predict(features, n_clusters)* – skaluje dane i uczy model KMeans
- *get_clustered_df()* – zwraca ramkę danych z przypisanymi klastrami

Klasa pochodna BudgetRevenueCluster

Klasteryzuje filmy na podstawie zmiennych „budget” i „revenue”

Klasa pochodna PopularityRatingCluster

Klasteryzuje filmy na podstawie zmiennych „popularity” i „vote_average”

Testy

Projekt zawiera zestaw testów jednostkowych oraz integracyjnych, których celem jest weryfikacja poprawności działania kluczowych elementów aplikacji. Testy zostały napisane z wykorzystaniem biblioteki *pytest* i obejmują zarówno logikę przetwarzania danych, analizy statystyczne, klasteryzację, jak i ładowanie danych.

test_movie_preprocessor.py

Zawiera testy klasy **MoviePreprocessor**, odpowiedzialnej za czyszczenie i transformację danych, których celem jest upewnienie się, że dane wejściowe są poprawnie przygotowane do dalszej analizy.

Zakres testów:

- testowanie wyciągania roku z daty - sprawdza, czy poprawne daty są prawidłowo konwertowane do kolumny *year*
- obsługa niepoprawnych danych wejściowych - weryfikuje, czy niepoprawna data nie powoduje błędu
- testowanie filtrowania po roku - sprawdza, czy dane są poprawnie filtrowane według minimalnego roku
- testowanie ekstrakcji danych kategorycznych - weryfikuje poprawne spłaszczanie danych zapisanych w formacie JSON

test_movie_analyzer.py

Zawiera testy klasy **MovieAnalyzer**, odpowiedzialnej za analizę statystyczną danych, których celem jest potwierdzenie poprawności agregacji danych wykorzystywanych w wykresach.

Zakres testów:

- sprawdzanie poprawności zliczania liczby filmów w poszczególnych latach
- weryfikowanie poprawności obliczania średniej oceny filmów dla danego roku
- testowanie, czy metoda poprawnie agreguje i zwraca najczęstsze gatunki

- sprawdzanie, czy brak kolumny *year* nie powoduje błędu

test_clusters.py

Zawiera testy klas **BudgetRevenueCluster** i **PopularityRatingCluster,** odpowiedzialnych za klasteryzację danych filmowych, których to testów celem jest potwierdzenie poprawnej logiki klasteryzacji i odporności na dane problematyczne.

Zakres testów:

- sprawdzenie czy po wykonaniu klasteryzacji do danych dodawana jest kolumna z etykietą klastra
- testowanie, czy algorytm KMeans tworzy oczekiwaną liczbę klastrów
- weryfikacja, czy rekordy z zerowymi wartościami są poprawnie usuwane

test_loader.py

Zawiera testy klasy **KaggleMoviesLoader,** odpowiedzialnej za ładowanie danych do aplikacji, których celem jest zapewnienie, że aplikacja działa poprawnie (również offline).

Zakres testów:

- sprawdzanie działania opcji awaryjnej – testowanie z wykorzystaniem mechanizmu mockowania, czy w przypadku braku dostępu do KaggleAPI dane są poprawnie wczytywane z lokalnego pliku CSV

test_integration.py

Zawiera test integracyjny, sprawdzający współdziałanie kilku klas jednocześnie, potwierdzający, że kluczowe elementy współpracują ze sobą poprawnie . Testowany jest przepływ:

1. Czyszczenie danych (**MoviePreprocessor**)
2. Analiza statystyczna (**MovieAnalyzer**)

Zakres testu:

- sprawdzenie poprawności wyciągania roku z daty
- sprawdzenie liczby filmów w danym roku
- sprawdzenie poprawności agregacji gatunków

Podsumowanie

Projekt stanowi kompletną aplikację analityczną typu EDA (Exploratory Data Analysis), pozwalającą użytkownikowi bez znajomości Pythona:

- eksplorować dane filmowe,
- filtrować je według własnych kryteriów,
- analizować trendy i zależności,
- wizualizować wyniki w czytelnej formie.

Zastosowanie klas w modułach pomocniczych poprawia czytelność i organizację kodu, umożliwia łatwe rozszerzanie funkcjonalności oraz pozwala oddzielić logikę aplikacji od warstwy interfejsu użytkownika. Zaś zaimplementowane testy obejmują wszystkie kluczowe moduły, sprawdzają różne przypadki oraz zwiększają niezawodność i stabilność aplikacji.